MLOPS Assignment 1

Naaz Verma

2025AF05094

Heart Disease Risk Prediction - End-to-End MLOps Pipeline

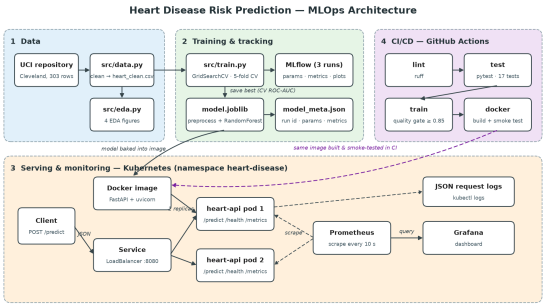
GitHub repository: https://github.com/naaz-bits/MLOPS-2025AF05094-Naaz-assignment1

Pipeline video:
https://drive.google.com/file/d/1NQay5J8RSeEPfjZwnst5nUMSHlH6FOjJ/view?usp=drive_link

1. Introduction

The project predicts whether a patient has heart disease (1) or not (0) from 13 clinical measurements, such as age, chest-pain type, cholesterol and maximum heart rate. It serves the prediction, with a confidence score, as a monitored API on Kubernetes. The data is the UCI Heart Disease Cleveland subset: 303 patients, with the 0–4 severity score `num` collapsed into a binary target. Tools: pandas and scikit-learn, MLflow, FastAPI, pytest and ruff, GitHub Actions, Docker, Kubernetes (Minikube), Prometheus and Grafana.

2. Architecture

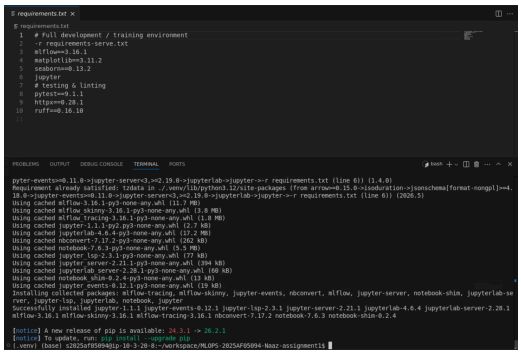


The pipeline has four layers, each handing one artifact to the next. **Data:** `src/data.py` downloads and cleans the dataset. **Training:** `src/train.py` tunes three models, logs them to

MLflow and saves the best as one Pipeline, `model.joblib`. **Serving:** a FastAPI container runs as 2 replicas on Kubernetes. **Operations:** GitHub Actions tests, retrains and smoke-tests on every push, while Prometheus and Grafana monitor the cluster. The key design choice is that the saved model contains its own preprocessing. Training, the notebook, CI and the API all call `model.predict()` on raw fields, so feature code cannot drift between training and serving.

3. Setup

git clone https://github.com/naaz-bits/MLOPS-2025AF05094-Naaz-assignment1.git && cd MLOPS-2025AF05094-Naaz-assignment1
python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
python -m src.data && python -m src.eda && python -m src.train # data -> figures -> models + MLflow
pytest -v && ruff check src api tests
mlflow ui --backend-store-uri sqlite:///mlflow.db # http://127.0.0.1:5000



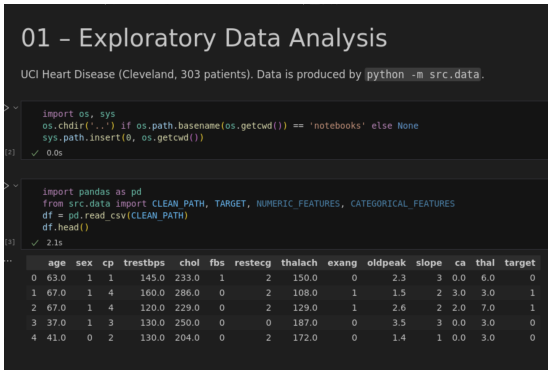
4. Data and EDA

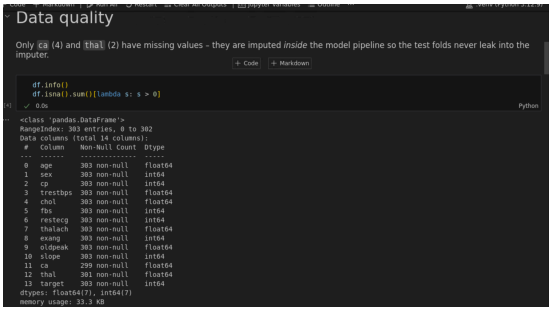
`src/data.py` downloads `processed.cleveland.data` from UCI and names the 14 columns. It reads `?` as missing, drops duplicate rows, binarises the target and nulls impossible zero values for cholesterol and blood pressure. Only `ca` (4 rows) and `thal` (2 rows) are missing. They are imputed inside the model pipeline, not here, so each cross-validation fold learns imputation from its own training data and nothing leaks from the test folds.

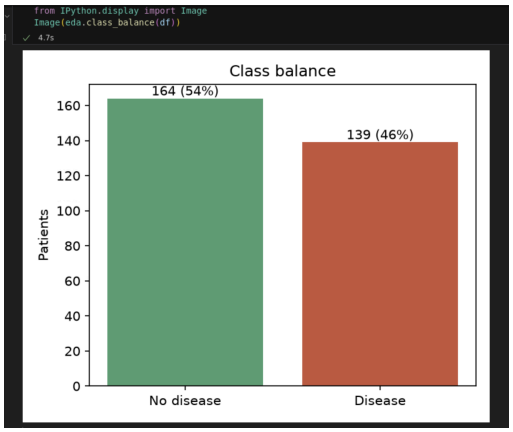
```
(.venv) (base) s2025af05094@ip-10  Open file in editor (ctrl + click) 325AF05094-Naaz-assignment1$ python -m src.data
INFO Raw data already present at /home/s2025af05094/workspace/MLOPS-2025AF05094-Naaz-assignment1/data/raw/processed.cleveland.data
INFO Wrote 303 rows x 14 cols to /home/s2025af05094/workspace/MLOPS-2025AF05094-Naaz-assignment1/data/processed/heart_clean.csv
INFO Missing values:
ca      4
thal    2
dtype: int64
INFO Class balance:
target
0    164
1    139
Name: count, dtype: int64
```

```
(.venv) (base) s2025af05094@ip-10-3-20-8:~/workspace/MLOPS-2025AF05094-Naaz-assignment1$ python -m src.eda
Matplotlib is building the font cache; this may take a moment.
          count    mean     std    min    25%    50%    75%    max
age       303.0   54.44    9.04   29.0   48.0   56.0   61.0   77.0
sex       303.0    0.68    0.47    0.0    0.0    1.0    1.0    1.0
cp        303.0    3.16    0.96    1.0    3.0    3.0    4.0    4.0
trestbps  303.0  131.69   17.60   94.0  120.0  130.0  140.0  200.0
chol      303.0  246.69   51.78  126.0  211.0  241.0  275.0  564.0
fbs       303.0    0.15    0.36    0.0    0.0    0.0    0.0    1.0
restecg   303.0    0.99    0.99    0.0    0.0    1.0    2.0    2.0
thalach   303.0  149.61   22.88   71.0  133.5  153.0  166.0  202.0
exang     303.0    0.33    0.47    0.0    0.0    0.0    1.0    1.0
oldpeak   303.0    1.04    1.16    0.0    0.0    0.8    1.6    6.2
slope     303.0    1.60    0.62    1.0    1.0    2.0    2.0    3.0
ca        299.0    0.67    0.94    0.0    0.0    0.0    1.0    3.0
thal      301.0    4.73    1.94    3.0    3.0    3.0    7.0    7.0
target    303.0    0.46    0.50    0.0    0.0    0.0    1.0    1.0
saved /home/s2025af05094/workspace/MLOPS-2025AF05094-Naaz-assignment1/reports/figures/class_balance.png
saved /home/s2025af05094/workspace/MLOPS-2025AF05094-Naaz-assignment1/reports/figures/numeric_histograms.png
saved /home/s2025af05094/workspace/MLOPS-2025AF05094-Naaz-assignment1/reports/figures/categorical_disease_rate.png
saved /home/s2025af05094/workspace/MLOPS-2025AF05094-Naaz-assignment1/reports/figures/correlation_heatmap.png
(.venv) (base) s2025af05094@ip-10-3-20-8:~/workspace/MLOPS-2025AF05094-Naaz-assignment1$
```

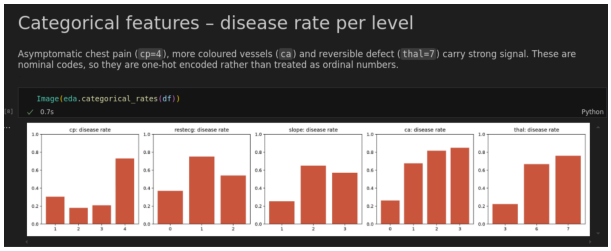
The classes are close to balanced: 164 patients without disease (54%) and 139 with (46%), so no resampling was needed. Patients with disease have a lower maximum heart rate (139 vs 158 bpm) and more ST depression (1.6 vs 0.6). Asymptomatic chest pain (`cp` = 4), more blocked vessels (`ca`) and reversible defect (`thal` = 7) carry the strongest signal. The highest correlations with the target are `thal` (0.53), `ca` (0.46) and `exang` (0.43); `chol` (0.09) and `fbs` (0.03) are the weakest. No two features are redundant, so all 13 are kept.

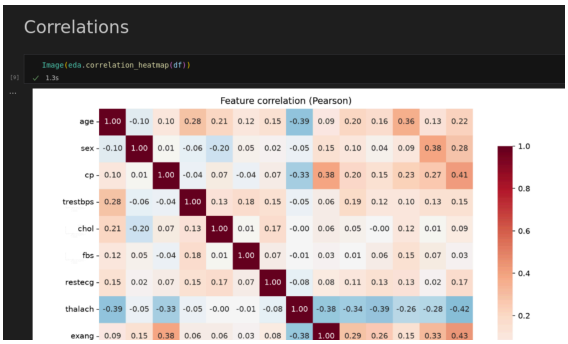










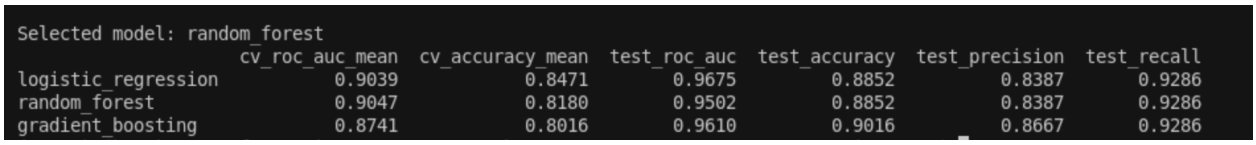


5. Features and models

`src/features.py` builds a `ColumnTransformer` with three branches. **Numeric** features (age, trestbps, chol, thalach, oldpeak) get median imputation, which is robust to outliers such as cholesterol 564, then standard scaling. **Categorical** codes (cp, restecg, slope, ca, thal) get most-frequent imputation and one-hot encoding, because they are unordered labels; `handle_unknown="ignore"` means an unseen code cannot crash the API. **Binary** features (sex, fbs, exang) are only imputed. The 13 inputs become 25 features, and the transformer and classifier are chained in one `Pipeline`.

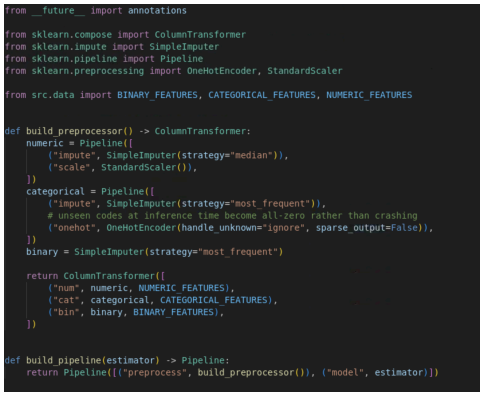
The data was split 80/20 (242/61), stratified, with seed 42. Each model was tuned with `GridSearchCV` (5-fold stratified CV, scored on ROC-AUC) and then re-scored with 5-fold CV on accuracy, precision, recall, F1 and ROC-AUC before one evaluation on the test set.

Model	Grid (size)	Best params	CV ROC-AUC	Test acc.	Test recall
Logistic Regression	C, class_weight (8)	C = 1, no weighting	0.9039	0.885	0.929
Random Forest	trees, depth, min leaf (12)	100, 5, 5	**0.9047**	0.885	0.929
Gradient Boosting	trees, learning rate, depth (8)	100, 0.05, 2	0.8741	0.902	0.929



```
Selected model: random_forest
                     cv_roc_auc_mean  cv_accuracy_mean  test_roc_auc  test_accuracy  test_precision  test_recall
logistic_regression           0.9039            0.8471        0.9675         0.8852          0.8387       0.9286
random_forest                 0.9047            0.8180        0.9502         0.8852          0.8387       0.9286
gradient_boosting             0.8741            0.8016        0.9610         0.9016          0.8667       0.9286
```

cv_accuracy_mean,cv_precision_mean,cv_recall_mean,cv_f1_mean,cv_roc_auc_mean,cv_accuracy_std,cv_precision_std,cv_recall_std,cv_f1_std,cv_roc_auc_std,test_accuracy,test_precision,test_recall,test_f1,test_roc_auc
logistic_regression,0.8471,0.8768,0.7834,0.8245,0.9039,0.01,0.0542,0.0459,0.0069,0.0138,0.8852,0.8387,0.9286,0.8814,0.9675
random_forest,0.818,0.8329,0.7644,0.7917,0.9047,0.0214,0.05,0.0894,0.0332,0.0208,0.8852,0.8387,0.9286,0.8814,0.9502
gradient_boosting,0.8016,0.79,0.783,0.7818,0.8741,0.0282,0.0501,0.0845,0.039,0.0244,0.9016,0.8667,0.9286,0.8966,0.961



```python
from __future__ import annotations

from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from src.data import BINARY_FEATURES, CATEGORICAL_FEATURES, NUMERIC_FEATURES


def build_preprocessor() -> ColumnTransformer:
    numeric = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ])
    categorical = Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        # unseen codes at inference time become all-zero rather than crashing
        ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
    ])
    binary = SimpleImputer(strategy="most_frequent")

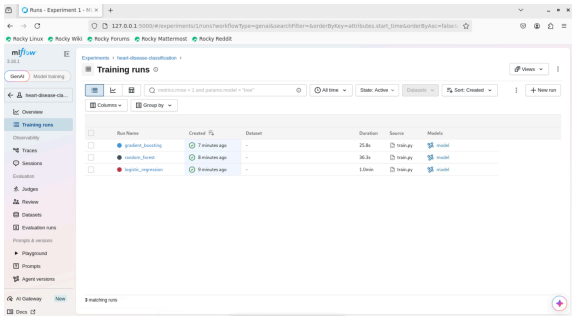
    return ColumnTransformer([
        ("num", numeric, NUMERIC_FEATURES),
        ("cat", categorical, CATEGORICAL_FEATURES),
        ("bin", binary, BINARY_FEATURES),
    ])

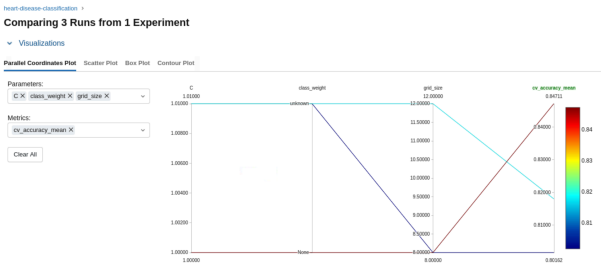

def build_pipeline(estimator) -> Pipeline:
    return Pipeline([("preprocess", build_preprocessor()), ("model", estimator)])
```

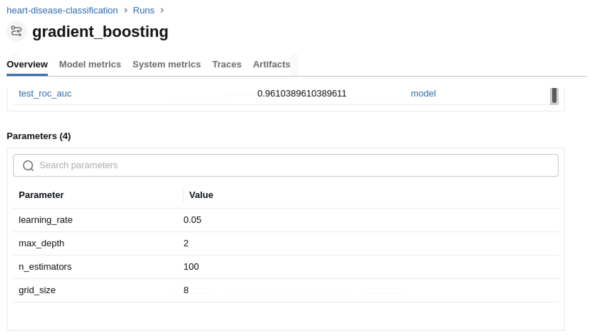
The winner is chosen on **CV ROC-AUC, not the test score**, because a 61-patient test set is too small to choose on. Random Forest and Logistic Regression are effectively tied: 0.9047 vs 0.9039, within one standard deviation of about 0.02. The rule picked Random Forest; Logistic Regression would be equally defensible where interpretability matters. ROC-AUC was used because it scores ranking across all thresholds. For screening, recall matters most, and all three models catch 93% of disease cases.

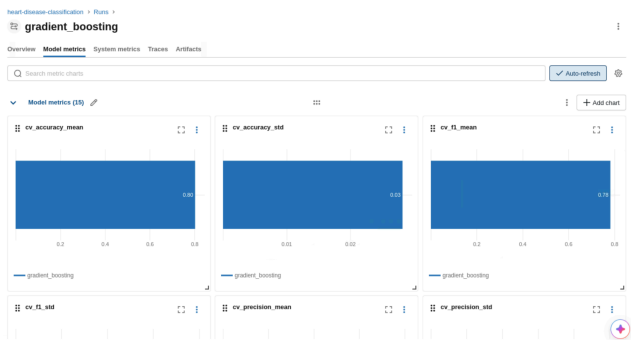
6. Experiment tracking

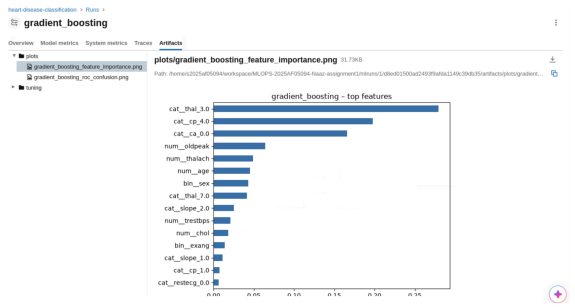
MLflow (experiment `heart-disease-classification`, SQLite store) records one run per model with the best parameters and grid size. Each run also logs 15 metrics (CV mean and std for the five metrics, plus five test metrics), a ROC curve with confusion matrix, a feature-importance plot, `cv_results.csv` covering every grid combination, and the fitted model. Keeping every grid combination means the tuning decision can be audited later. The feature-importance plot agrees with the EDA: `thal`, `cp` = 4 and `ca` rank highest.

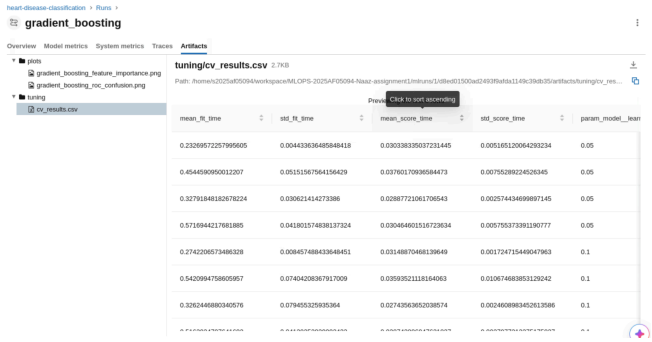








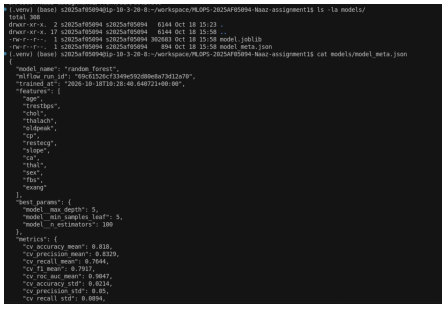




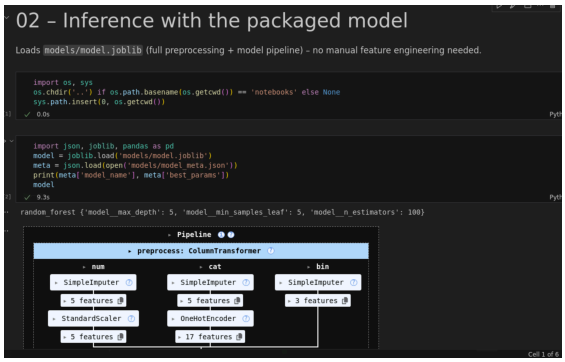
7. Packaging and reproducibility

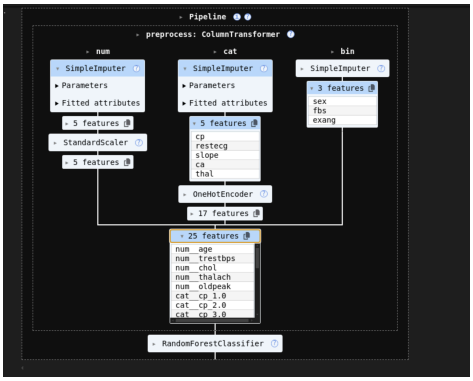
The winning Pipeline is saved as `models/model.joblib` and also in MLflow format. `model_meta.json` records the run ID, timestamp, feature order, parameters and metrics, and the API returns it at `/info`, so any prediction can be traced to its training run. Reproducibility comes from:

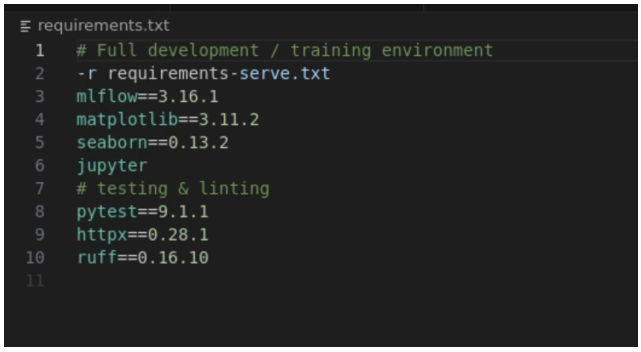
- pinned versions in `requirements.txt` and `requirements-serve.txt`,
- seed 42 everywhere,
- preprocessing stored inside the model,
- committed raw and cleaned data,
- CI retraining from scratch on a clean runner on every push.









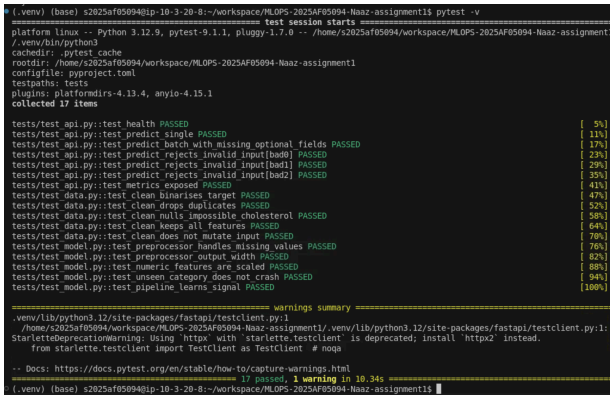


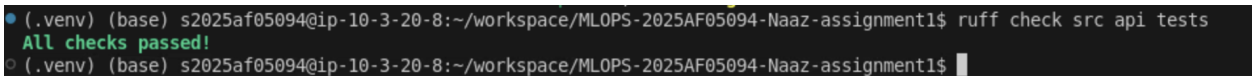
8. CI/CD and testing

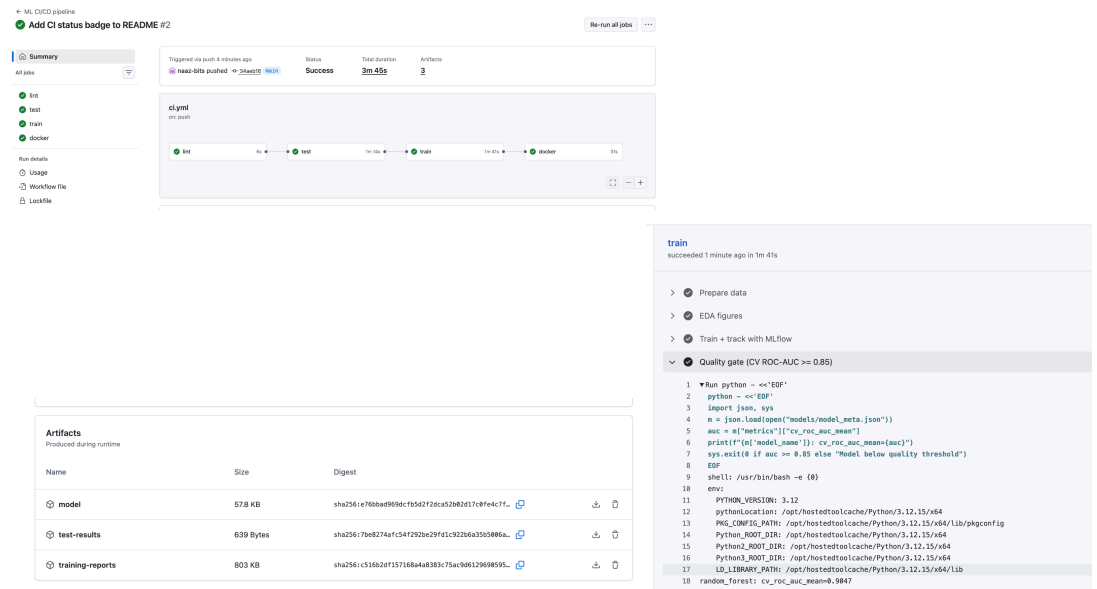
17 pytest tests cover three layers. `test_data.py` (5) checks target binarisation, duplicate removal and impossible-value handling. `test_model.py` (5) checks imputation, output width, scaling, unseen categories and that the pipeline learns. `test_api.py` (7) checks `/health`, single and batch prediction, and three invalid inputs returning 422. All pass in about 10 s and ruff reports no issues

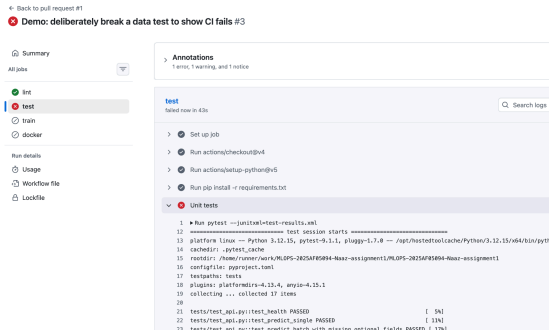
`.github/workflows/ci.yml` runs on every push, pull request and manual trigger. Four jobs run in sequence, each only if the previous one passed: **lint** (ruff) → **test** (pytest, JUnit report uploaded) → **train** (data, EDA and training, then a **quality gate** that fails if CV ROC-AUC <

0.85; model, logs and MLflow store uploaded) → **docker** (build the image, start it, call `/predict` with `sample_request.json`, and assert two results). The green run took 3 min 45 s and stored three artifacts . To prove the pipeline fails safely, a PR broke one test assertion: `test` failed and `train` and `docker` were skipped, so nothing was built from broken code.





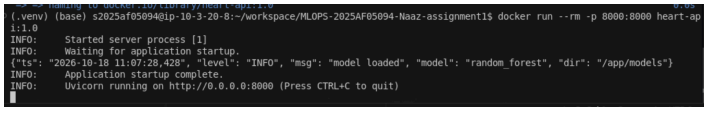




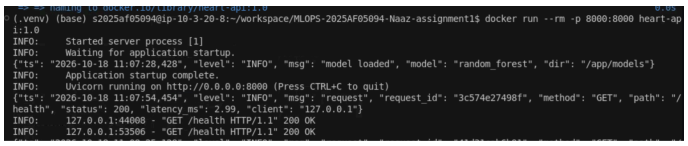
9. Container and API

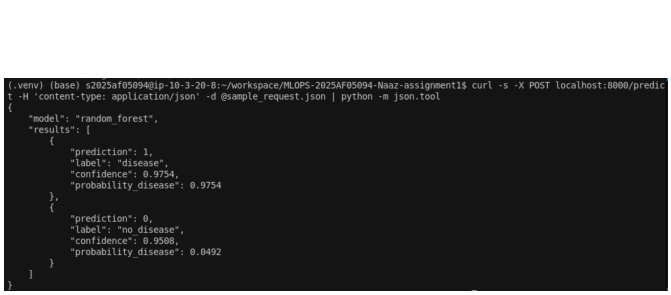
The FastAPI service (`api/main.py`) loads the model once and exposes `POST /predict` (one patient or a list → prediction, label, confidence, probability of disease), `/health`, `/info` and `/metrics`. Pydantic validates every field's type and range, so bad input such as `age` = −5 gets an HTTP 422 naming the field instead of a silent wrong answer. `ca` and `thal` are optional and imputed. The image uses `python:3.12-slim` with only serving dependencies, copies just `src/`, `api/` and the model, runs as a non-root user, and has a `HEALTHCHECK`. For the two sample patients, the container returns *disease* (confidence 0.975) and *no disease* (0.951)

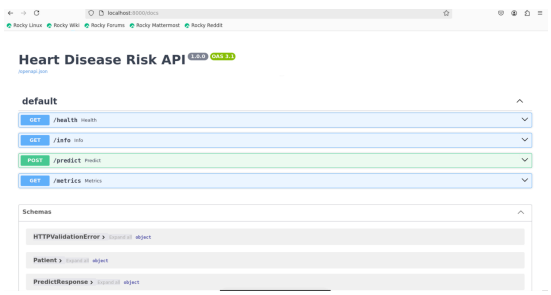
docker build -t heart-api:1.0 . && docker run --rm -p 8000:8000 heart-api:1.0
curl -s -X POST localhost:8000/predict -H 'content-type: application/json' -d @sample_request.json

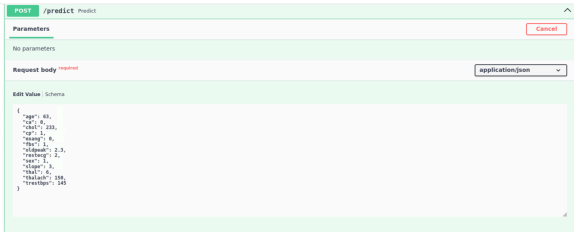


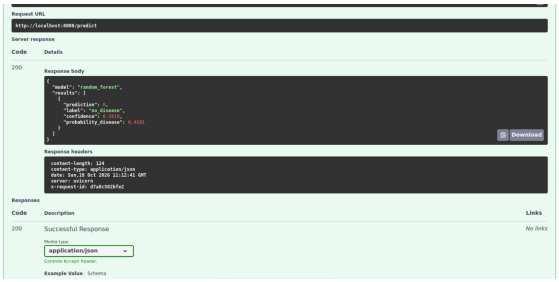


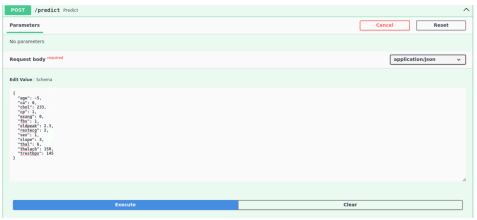














10. Kubernetes deployment

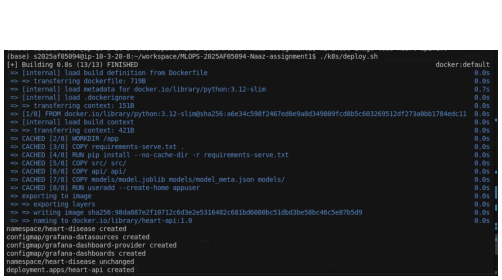
The API runs on Minikube (Kubernetes v1.37) on the Prayogshala VM, in namespace `heart-disease`. Manifests in `k8s/` define a Deployment with **2 replicas**, a rolling update with `maxUnavailable: 0`, readiness and liveness probes on `/health`, and CPU and memory limits. They also define a **LoadBalancer** Service (8080 → 8000), Prometheus, and a pre-provisioned Grafana. Two replicas plus readiness probes mean updates cause no downtime, and no pod gets traffic before its model has loaded. `k8s/deploy.sh` builds the image, applies everything and waits for the rollout

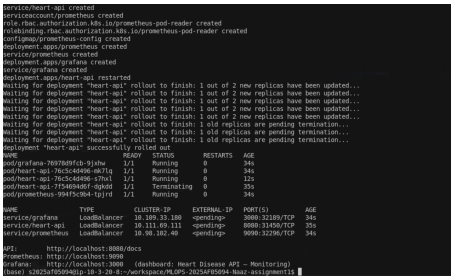
Access (local, no public URL). Minikube has no cloud load balancer, so `EXTERNAL-IP` stays `<pending>` . Use port-forward, or `minikube tunnel`:

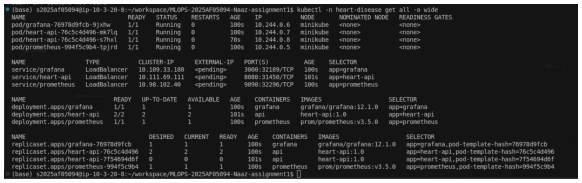
minikube start --driver=docker && minikube image load heart-api:1.0 && ./k8s/deploy.sh

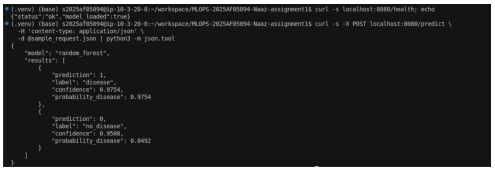
kubectl -n heart-disease port-forward svc/heart-api 8080:8080 & # also prometheus 9090, grafana 3000

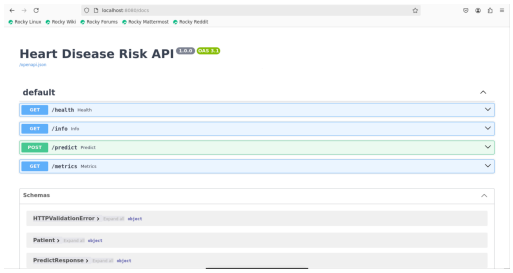
curl -s localhost:8080/health # Swagger: http://localhost:8080/docs









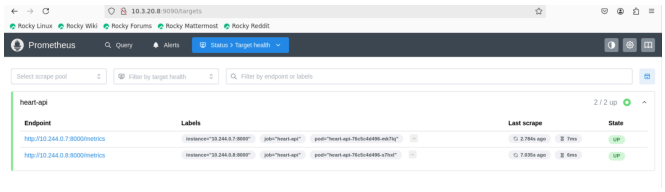


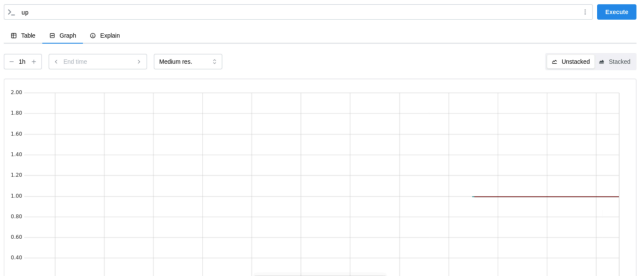
11. Monitoring and logging

Middleware writes one JSON log line per request with the time, a `request_id` (also returned in the `X-Request-ID` header), method, path, status, latency and client. JSON makes the logs filterable by field, for example every request with status 422. The API exposes four Prometheus metrics: request count and latency per path, plus two model metrics, predictions per class and prediction confidence. A shift in the class mix or falling confidence is an early sign of data drift, even when every request succeeds. Prometheus discovers pods through Kubernetes annotations and scrapes every 10 s, so new replicas are monitored automatically; both pods show as UP

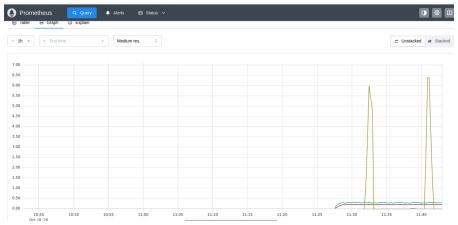
Grafana is provisioned from files, so the dashboard exists as soon as it is deployed. Under generated load it showed 1.17K predictions, 96.3% mean confidence, 2 healthy pods, request rate, p50/p95 latency, and predictions by class and status code. The error-rate panel counts only 5xx errors. There were none, so it shows "No data"; the query now returns 0% instead.

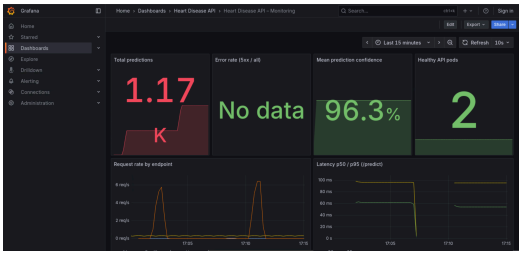






sum by (path) (rate(api_requests_total[1m]))







12. Conclusion and links

The result is a heart-disease classifier with CV ROC-AUC 0.90 and test recall 0.93. It is served as a validated container on Kubernetes, tracked in MLflow, retested and retrained by CI on every push, and monitored live. The main lesson was to ship preprocessing inside the model, which removed training/serving mismatch and made the CI smoke test a real end-to-end check.

Limitations and next steps. 303 patients from one hospital is a small sample, and the 61-patient test set gives noisy estimates. Deployment is local with no public URL, and there is no ground-truth feedback to measure live accuracy. Next steps: push the image to a registry and

deploy to a managed cluster, add Grafana alerts on error rate and confidence drift, and promote models through the MLflow Model Registry.

Links and deliverables.

Item	Location
Code repository	https://github.com/naaz-bits/MLOPS-2025AF05094-Naaz-assignment1
CI/CD runs	https://github.com/naaz-bits/MLOPS-2025AF05094-Naaz-assignment1/actions
All 50 screenshots	`screenshots/` in the repository
Cleaned dataset / download script	`data/processed/heart_clean.csv` / `src/data.py`
Notebooks	`notebooks/01_eda.ipynb`, `notebooks/02_inference.ipynb`
Deployment manifests	`k8s/`
Pipeline video	https://drive.google.com/file/d/1NQay5J8RSeEPfjZwnst5nUMSHlH6FOjJ/view?usp=drive_link
API access	local only: see section 10